Actividad Obligatoria II — Simulación con FastAPI de CoAP, WebSocket y MQTT usando HiveMQ

Licenciatura en Ciberdefensa — Dispositivos Remotos e Internet de las Cosas (FADENA / UNDEF) — 2026

Autor: Gustavo Courault — Fork: `github.com/gcourault/iot_repo`

---

1. Entorno y procedimiento

Ítem	Valor
Sistema	Debian 12.15, kernel 6.1.0-53-amd64
Python / gestor	3.12.14 / `uv` 0.12.13
Librerías	fastapi 0.126.0, uvicorn 0.40.0, paho-mqtt 2.1.0, websockets 15.0.1, aiocoap 0.4.17, pydantic 2.12.5
Wireshark / tshark / dumpcap	4.0.17
Bróker	**`broker.hivemq.com`** (clúster público de HiveMQ, sin bróker local)
Tópico	`fadena/undef/gcourault/telemetria`
Interfaces capturadas	`eno1` (Ethernet, 192.168.1.24/23, gateway 192.168.1.1) y `lo` (loopback)

A diferencia de la Actividad 1, **no se usa Mosquitto local**: todo el tráfico MQTT sale a Internet hacia el clúster público de HiveMQ y vuelve.

1.1 Scripts

Script	Rol
`lectura.py`	Servidor IoT FastAPI: HTTP `POST /telemetria`, WebSocket `/ws`, puente MQTT (HiveMQ → WebSocket) y gateway `GET /coap/temp` (HTTP → CoAP)
`sensor_mq.py`	Publicador MQTT hacia `broker.hivemq.com:1883`
`lectura_mq.py`	Suscriptor MQTT + API HTTP en el puerto 8001
`hivemq_ws_client.py`	Suscriptor MQTT **sobre WebSocket** (`ws://broker.hivemq.com:8000/mqtt`)
`ws_client.py` / `ws_listener.py`	Cliente emisor y oyente del WebSocket de FastAPI
`coap_server.py` / `coap_client.py`	Servidor y cliente CoAP sobre UDP/5683

Todos leen su configuración de variables de entorno (`MQTT_BROKER`, `MQTT_PORT`, `MQTT_TOPIC`, `SENSOR_COUNT`, `WS_PORT`, `WS_SECONDS`...), de modo que el mismo código sirve para un bróker local o para el público.

1.2 Experimentos y capturas

En cada experimento se inició `dumpcap` **antes** de los scripts, con filtro de captura por puerto:

#	Experimento	Interfaz	Filtro	Archivo	Paquetes
A	MQTT → HiveMQ (TCP 1883)	`eno1`	`tcp port 1883`	`capturas/act2/mqtt_hivemq_eno1.pcapng`	58
B	MQTT sobre WebSocket → HiveMQ (TCP 8000)	`eno1`	`tcp port 8000`	`capturas/act2/mqtt_ws_hivemq_eno1.pcapng`	30
C	FastAPI: HTTP + WebSocket + puente MQTT	`lo`	`tcp port 8000`	`capturas/act2/fastapi_ws_lo.pcapng`	170
D	CoAP + gateway HTTP→CoAP	`lo`	`udp port 5683 or tcp port 8000`	`capturas/act2/coap_lo.pcapng`	26
E	Tres suscriptores simultáneos al mismo tópico	`eno1`	`tcp port 1883 or tcp port 8000`	`capturas/act2/multisub_eno1.pcapng`	97

Los logs de cada script quedaron junto a las capturas en `capturas/act2/*.log` y las salidas de la API en `capturas/act2/*.json`.

> **Nota sobre el punto 4 del manual.** La suscripción "desde Internet" se hizo con `hivemq_ws_client.py`, que replica exactamente lo que hace el cliente web oficial de HiveMQ: abre un WebSocket contra `broker.hivemq.com:8000`, path `/mqtt`, subprotocolo `mqtt`, y habla MQTT dentro de ese WebSocket. El tráfico en la red es idéntico al que genera la página (experimento B), con la ventaja de que queda registrado en una captura analizable. No hay, por lo tanto, captura de pantalla del cliente web, sino la traza de red de la misma conversación.

---

2. Funcionamiento

2.1 Publicación hacia el bróker público

`sensor_mq.py` publicó 8 lecturas (`capturas/act2/mqtt_sensor.log`):

```
Conectando a broker público broker.hivemq.com:1883 (client_id=sensor-act2)...
✓ CONNACK de broker.hivemq.com:1883 → Success (session_present=False)
  (CONNACK recibido en 0.46 s)
Publicando en el tópico 'fadena/undef/gcourault/telemetria' con QoS 0 cada 2.0s
[001] Publicado en fadena/undef/gcourault/telemetria (81 bytes): {"sensor_id": "S-001", "valo
r": 27.64, "timestamp": "2026-09-21T09:39:26.789487"}
...
Total publicado: 8 mensajes
```

2.2 Recepción desde Internet

`lectura_mq.py`, suscrito al mismo tópico, recibió **las 8 lecturas** después de que dieran la vuelta por Frankfurt (`capturas/act2/mqtt_lectura.log`, historial en `capturas/act2/mqtt_historial.json`):

```
Conectado a broker.hivemq.com:1883 con código: Success
Suscrito al tópico: fadena/undef/gcourault/telemetria
[HIVEMQ] Recibido en fadena/undef/gcourault/telemetria: {"sensor_id": "S-001", "valor": 27.64,
...}
```

2.3 Suscripción por WebSocket (equivalente al cliente web)

`hivemq_ws_client.py` recibió las 6 publicaciones del experimento B (`capturas/act2/mqtt_ws_cliente.log`):

```
✓ Conectado por ws://broker.hivemq.com:8000/mqtt → Success
✓ Subscribe a 'fadena/undef/gcourault/telemetria' (equivale al botón Subscribe del cliente web)
[09:40:08.402] fadena/undef/gcourault/telemetria (qos 0, 81 bytes): {"sensor_id": "S-001", "val
or": 28.57, ...}
...
Total de mensajes recibidos por WebSocket: 6
```

2.4 Los tres protocolos convergiendo en FastAPI

En el experimento C, el mismo proceso FastAPI recibió datos por HTTP, por WebSocket y por MQTT (desde HiveMQ), y difundió **todo** a los clientes WebSocket conectados (`capturas/act2/fastapi_ws_listener.log`):

```
[09:40:54.195] origen=websocket  {"temperature": 23.11, "timestamp": "2026-09-21T09:40:54.194288"}
[09:41:03.205] origen=http       {"sensor_id": "S-001", "valor": 20.82}
[09:41:10.874] origen=mqtt       {"sensor_id": "S-001", "valor": 21.01, "timestamp": "2026-09-21T09:41:10.636346"}
```

`GET /eventos` confirma el recuento: **12 eventos = 5 WebSocket + 4 HTTP + 3 MQTT**. La línea `origen=mqtt` es la cadena completa: sensor local → Internet → bróker en Alemania → Internet → FastAPI → navegador. Publicado a las `09:41:10.636`, mostrado en el panel a las `09:41:10.874`: **238 ms de vuelta al mundo.**

2.5 CoAP y gateway HTTP→CoAP

`coap_client.py` envió 6 lecturas y `coap_server.py` las confirmó con `2.04 Changed`. El gateway de FastAPI tradujo una petición HTTP en una CoAP (`capturas/act2/coap_gateway.json`):

```
{"status":"ok","codigo_coap":"2.05 Content","dato":{"temperature":25.37,"timestamp":"2026-09-21T09:42:54.811269"}}
```

2.6 Tres suscriptores simultáneos

Para comprobar el desacoplamiento del modelo publicador/bróker/suscriptor se lanzaron a la vez los tres tipos de consumidor contra el mismo tópico y se publicaron 5 lecturas:

Suscriptor	Transporte	Mensajes recibidos	Log
`lectura_mq.py`	MQTT sobre TCP 1883	**5 de 5**	`capturas/act2/multisub_1_tcp.log`
Puente dentro de `lectura.py`	MQTT sobre TCP 1883	**5 de 5**	`capturas/act2/multisub_2_fastapi.log`
`hivemq_ws_client.py`	MQTT sobre WebSocket 8000	**5 de 5**	`capturas/act2/multisub_3_ws.log`

El publicador no fue modificado ni informado de ninguno de ellos.

---

3. Análisis por capas de red

3.1 Capa 7 — Aplicación

MQTT sobre TCP (experimento A). Secuencia completa observada en la captura:

Trama	Sentido	Mensaje MQTT	Tamaño de trama
4	cliente → bróker	CONNECT (msgtype 1)	92 bytes
6	bróker → cliente	CONNACK (2)	70 bytes
8	cliente → bróker	SUBSCRIBE (8) `fadena/undef/gcourault/telemetria`	106 bytes
10	bróker → cliente	SUBACK (9), *Granted QoS 0*	71 bytes
19	publicador → bróker	PUBLISH (3)	184 bytes
20	bróker → suscriptor	PUBLISH (3), mismo payload	184 bytes
51	cliente → bróker	DISCONNECT (14)	68 bytes

El payload viaja **en texto plano**. Wireshark lo muestra campo por campo y el volcado hexadecimal se decodifica sin esfuerzo:

```
7b 22 73 65 6e 73 6f 72 5f 69 64 22 3a 20 22 53 2d 30 30 31 22 ...
→ {"sensor_id": "S-001", "valor": 27.64, "timestamp": "2026-09-21T09:39:26.789487"}
```

Con QoS 0 no hay PUBACK: un único PUBLISH por lectura en cada sentido. Los `PINGREQ` / `PINGRESP` mantienen viva la sesión (keepalive de 60 s).

MQTT dentro de WebSocket (experimento B). El handshake es HTTP:

```
GET /mqtt HTTP/1.1
Host: broker.hivemq.com:8000
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: ZiH2Oh9mRI2utDlsiRrk6g==
Sec-Websocket-Version: 13
Sec-Websocket-Protocol: mqtt

HTTP/1.1 101 Switching Protocols
upgrade: websocket
connection: upgrade
sec-websocket-accept: dtlfbfogBMV9FZPpEUmjRgwkwVo=
sec-websocket-protocol: mqtt
```

A partir del `101` la conexión deja de ser HTTP y transporta tramas WebSocket. Decodificando el payload TCP de una de ellas se ve el **triple anidamiento**:

```
82 76 | 30 74 | 00 21 "fadena/undef/gcourault/telemetria" | {"sensor_id": "S-001", ...}
└─ WS ┘ └ MQTT ┘ └──────────── topic (33 bytes) ────────────┘ └──────── payload JSON ────────┘

Cabecera WebSocket: 82 76 → FIN=1, opcode=2 (binario), MASK=0, longitud 118
MQTT dentro:        30 74 → msgtype 3 (PUBLISH), remaining length 116
```

Es decir: **Ethernet → IPv4 → TCP → WebSocket → MQTT → JSON**. El subprotocolo `mqtt` negociado en el handshake es lo que le dice al bróker que interprete los bytes del WebSocket como MQTT. Las tramas cliente→servidor llevan `MASK=1` (el RFC 6455 obliga al cliente a enmascarar); las del servidor, `MASK=0`.

WebSocket de FastAPI (experimento C). El handshake añade una extensión que MQTT no negocia:

```
Sec-WebSocket-Extensions: permessage-deflate; client_max_window_bits
```

El servidor la acepta, y el efecto se mide en la captura: la primera lectura ocupa 66 bytes de payload WebSocket, la segunda 21 y la tercera 18. Al ser JSON casi idéntico, el diccionario de *deflate* lo comprime a una tercera parte. HTTP no puede hacer esto porque cada petición abre una conexión nueva y empieza de cero.

CoAP (experimento D). Formato REST sobre UDP, sin texto de cabecera:

Trama	Tipo	Código	MID	Token	Trama	UDP
1	CON (0)	POST (0.02)	3133	`3c10`	149 bytes	95 bytes
2	ACK (2)	2.04 Changed (68)	3133	`3c10`	106 bytes	52 bytes
18	CON (0)	GET (0.01)	21715	`358f`	83 bytes	29 bytes
19	ACK (2)	2.05 Content (69)	21715	`358f`	134 bytes	80 bytes

La fiabilidad la aporta el propio CoAP, no la capa de transporte: el tipo `CON` (*confirmable*) y el *Message ID* permiten detectar pérdidas y retransmitir; el *token* casa petición con respuesta. Una petición `GET` completa ocupa **21 bytes de CoAP** (29 de UDP menos 8 de cabecera): cabecera de 4 bytes, token de 2 y la opción Uri-Path `temp`. Los códigos son numéricos (`2.04`, `2.05`), no cadenas como `"200 OK"`.

3.2 Capa 4 — Transporte

Protocolo	Transporte	Puerto	Conexión
MQTT	TCP	1883	una persistente por cliente
MQTT sobre WebSocket	TCP	8000	una persistente
WebSocket (FastAPI)	TCP	8000	una persistente
HTTP	TCP	8000	**una nueva por cada lectura**
CoAP	**UDP**	5683	sin conexión

Coste real de una lectura, medido en las capturas:

Protocolo	Tramas por lectura	Bytes por lectura	Dato útil	Eficiencia
HTTP (`lo`)	14	1.363	37 B	2,7 %
WebSocket con deflate (`lo`)	4	~331	66 B → 21 B comprimidos	~20 %
MQTT QoS 0 (Internet)	2 (PUBLISH + ACK de TCP)	250	81 B	32 %
CoAP POST (`lo`)	**2**	**255**	65 B	25 %
CoAP GET (`lo`)	**2**	**217**	21 B de CoAP	—

HTTP paga un *three-way handshake*, cuatro segmentos de datos y un cierre `FIN` **por cada lectura**: 14 tramas y 1.363 bytes para transportar 37 bytes de JSON. La captura del experimento C lo muestra con toda claridad: cuatro conversaciones TCP distintas (puertos efímeros 44000, 44010, 44020, 44030), cada una de 14 tramas y 1.363 bytes, frente a **una sola** conversación WebSocket (puerto 44748) que se mantuvo abierta 34,0 s y cursó todo el tráfico del panel en 46 tramas.

CoAP es el caso extremo: **dos datagramas y nada más**. No hay handshake, no hay `FIN`, no hay ACK de transporte. El tiempo de respuesta medido fue de **0,63 a 1,36 ms**, porque no hay nada que establecer.

MSS negociado: **1.460 bytes** hacia Internet (Ethernet, MTU 1500) frente a los 65.495 del loopback de la Actividad 1 — la diferencia entre hablar con el propio kernel y hablar con Alemania.

Latencia a través del bróker público. Midiendo, en la misma captura, el instante en que sale el PUBLISH del publicador y el instante en que entra el PUBLISH al suscriptor:

```
publicaciones: 8   entregas: 8
latencias (ms): 235,2  235,2  235,4  234,9  235,0  235,3  235,0  235,2
mínimo 234,9   máximo 235,4   media 235,1
```

Una desviación de medio milisegundo sobre 235 ms: la ruta y la carga del clúster fueron completamente estables durante la prueba. Esos 235 ms son el precio de poner el bróker en Internet; en la Actividad 1, contra Mosquitto en `localhost`, el mismo recorrido costaba microsegundos.

3.3 Capa 3 — Red

Ésta es la diferencia de fondo con la Actividad 1: allí todo era `127.0.0.1 → 127.0.0.1`; aquí el tráfico **sale de la máquina**.

Extremo	Dirección	TTL
Origen (este equipo)	`192.168.1.24` (privada, RFC 1918)	64 al salir
HiveMQ, conexión del publicador	`18.185.58.114` (AWS `eu-central-1`, Frankfurt)	246 al llegar
HiveMQ, conexión del suscriptor	`63.186.26.177`	246 al llegar
HiveMQ, conexión WebSocket	`18.185.58.114`	246 al llegar

Un TTL de llegada de 246 sobre un valor inicial de 255 significa **9 saltos de router** entre el bróker y este equipo. Como la IP de origen que ve HiveMQ no es `192.168.1.24` sino la pública del enlace, hay además un **NAT** en el router doméstico que no aparece en la captura local.

Hallazgo: el bróker público es un clúster. `broker.hivemq.com` resuelve a seis direcciones —tres IPv4 y tres IPv6—:

```
18.185.214.85   18.185.58.114   63.186.26.177
2a01:578:13::12b9:3a72   2a01:578:13::3fba:1ab1   2a01:578:13::12b9:d655
```

En el experimento A el publicador y el suscriptor, lanzados con segundos de diferencia, **fueron a parar a nodos distintos** (`18.185.58.114` y `63.186.26.177`) y aun así el mensaje llegó: el clúster replica internamente las suscripciones. El experimento E lo muestra todavía mejor: cuatro conexiones TCP simultáneas repartidas sobre **tres nodos diferentes** del clúster, todas con el mismo tópico.

```
192.168.1.24:44613 <-> 63.186.26.177:8000    (cliente MQTT sobre WebSocket)
192.168.1.24:57083 <-> 18.185.214.85:1883    (puente de FastAPI)
192.168.1.24:55905 <-> 18.185.58.114:1883    (lectura_mq.py)
192.168.1.24:59805 <-> 63.186.26.177:1883    (sensor_mq.py, publicador)
```

Esto es exactamente lo que no puede hacer un bróker local, y es la base de la escalabilidad de MQTT.

También conviene notar que, aunque hay registros AAAA, todas las conexiones salieron por **IPv4**: la red local no tiene conectividad IPv6 global, así que el resolutor y `connect()` acabaron cayendo a IPv4.

3.4 Capa 2 — Enlace

	Actividad 1 (loopback)	Actividad 2 (Internet)
MAC origen	`00:00:00:00:00:00`	`f4:b5:20:4e:f1:9b` (NIC de este equipo)
MAC destino	`00:00:00:00:00:00`	`d0:07:ca:75:69:f0` (gateway 192.168.1.1)
EtherType	0x0800	0x0800 (IPv4)

En las tramas hacia HiveMQ, la MAC destino **no es la del bróker**: es la del router. Las direcciones MAC solo tienen sentido dentro del segmento Ethernet local; a partir del router, cada salto reescribe el encabezado de capa 2 mientras la IP destino (`18.185.58.114`) permanece intacta de extremo a extremo. `ip neigh` confirma la correspondencia:

```
192.168.1.1 dev eno1 lladdr d0:07:ca:75:69:f0 REACHABLE
```

En el experimento D, sobre `lo`, se ve el caso contrario: MACs a cero y 14 tramas IPv6 puras entre `::1` y `::1`.

---

4. Comparación de protocolos

Característica	HTTP	WebSocket	MQTT	CoAP
Transporte	TCP	TCP	TCP	**UDP**
Puerto	8000	8000	1883 / 8883 / 8000 (ws)	5683
Arquitectura	cliente-servidor	cliente-servidor directo	**publicador-bróker-suscriptor**	cliente-servidor
Conexión	una por petición	persistente	persistente	sin conexión
Dirección	petición/respuesta	bidireccional	unidireccional por mensaje, N a N	petición/respuesta
Desacoplamiento	ninguno	ninguno	total (por tópico)	ninguno
Fiabilidad	la de TCP	la de TCP	TCP + QoS 0/1/2	**en la capa 7** (CON/ACK, MID)
Cabecera mínima	~200 B de texto	2–14 B binarios	**2 B**	**4 B**
Coste medido por lectura	1.363 B	~331 B	250 B	255 B
Escalabilidad	media	media	**alta**	alta
Uso típico en IoT	integración puntual	paneles en tiempo real	**telemetría masiva**	**dispositivos limitados**

TCP frente a UDP. TCP garantiza entrega y orden, pero cobra por ello: handshake, ACKs, retransmisiones y mantenimiento de estado por conexión. Para un sensor alimentado a pilas que manda una lectura cada minuto, ese estado es un lujo. CoAP renuncia a él y reconstruye en la capa 7 solo lo que necesita: si el mensaje es `CON`, el receptor debe contestar `ACK` con el mismo *Message ID*, y el emisor retransmite si no llega. Se paga fiabilidad solo cuando hace falta (existe el tipo `NON` para lo que no la necesita).

Modelos de comunicación. WebSocket y CoAP son *punto a punto*: el cliente debe conocer y alcanzar al servidor. MQTT interpone un bróker, y eso cambia todo: el sensor no sabe quién lo escucha, los suscriptores no saben quién publica, y ninguno de los dos necesita ser alcanzable desde el otro. El experimento E lo demuestra: tres suscriptores **simultáneos** de naturaleza distinta —`lectura_mq.py` por MQTT/TCP, el puente dentro de FastAPI y `hivemq_ws_client.py` por MQTT/WebSocket— recibieron **las 5 publicaciones cada uno** sin que `sensor_mq.py` supiera nada de ellos ni cambiara una línea. Es también lo que permite que un dispositivo detrás de NAT publique hacia el mundo: solo abre una conexión saliente.

MQTT sobre TCP frente a MQTT sobre WebSocket. El mismo protocolo, dos transportes. El WebSocket añade el handshake HTTP inicial y 2–6 bytes por trama, pero atraviesa proxies y cortafuegos corporativos que solo dejan pasar 80/443, y es la **única** forma de hablar MQTT desde JavaScript en un navegador. Por eso el cliente web de HiveMQ usa el puerto 8000 y no el 1883.

---

5. Incidencias encontradas y su resolución

Tres fallos aparecieron durante las pruebas. Los tres resultaron instructivos y están corregidos en el código entregado.

5.1 Mensajes perdidos por publicar antes del CONNACK

Síntoma. El puente MQTT de FastAPI se conectaba, se suscribía (`SUBACK: Granted QoS 0`) y no recibía nada, mientras el publicador informaba de 3 publicaciones correctas. Ocurría de forma intermitente.

Diagnóstico. La captura de red desmintió al log. En ella se ve el CONNECT del publicador en t=8,68 s, sus tres PUBLISH en t=8,91 / 10,68 / 12,68 s… y el **CONNACK del bróker recién en t=13,68 s**. El bróker público tardó cinco segundos en contestar al CONNECT, y el cliente había publicado —y desconectado— antes. En otra prueba el CONNACK tardó **7,08 segundos**.

La causa estaba en el código heredado de la Actividad 1: `connect()` de paho solo establece el TCP y devuelve el control de inmediato; no espera el CONNACK. Contra Mosquitto en `localhost` la diferencia era de microsegundos y nunca se notó. Contra un bróker público saturado, sí. Publicar antes del CONNACK viola además el propio protocolo: el servidor debe procesar el CONNECT antes que cualquier otro paquete, y puede descartar lo que llegue antes.

Corrección. `sensor_mq.py` bloquea ahora en un `threading.Event` que `on_connect` activa, e informa del tiempo que tardó:

```
✓ CONNACK de broker.hivemq.com:1883 → Success (session_present=False)
  (CONNACK recibido en 0.46 s)
```

Se añadió además un segundo de margen antes del DISCONNECT, para que el bróker curse el último PUBLISH. Tras el arreglo, las 8 publicaciones del experimento A se recibieron **8 de 8**.

5.2 CoAP contra un puerto cerrado en IPv6

Síntoma. `Failed to send CoAP request: Network error: NetworkError` en todas las peticiones.

Diagnóstico. La captura mostró los datagramas saliendo hacia `::1` mientras `coap_server.py` estaba atado a `127.0.0.1`. El cliente resuelve `localhost` y obtiene primero `::1`; los datagramas llegaban a un puerto cerrado. Es el mismo patrón que en la Actividad 1, donde

uvicorn escuchaba solo en IPv4 y el cliente HTTP recibía `RST` por IPv6 antes de caer a IPv4 —con la diferencia de que UDP no tiene reintento equivalente.

Corrección. El servidor CoAP se ata a `::` (doble pila IPv4/IPv6).

5.3 Identificadores de cliente repetidos en un bróker público

En un bróker compartido con el mundo entero, un `client_id` fijo como `fastapi-bridge` es un riesgo real: el estándar MQTT obliga al bróker a **desconectar al cliente que ya estuviera conectado** con ese identificador. Cualquier otra persona usando el mismo nombre echa nuestra sesión. Todos los scripts generan ahora un sufijo aleatorio, y `lectura.py` registra las desconexiones (`on_disconnect`) para que el problema sea visible si vuelve a ocurrir.

---

6. Conclusiones

1. **El bróker público funciona y se nota.** Los datos de un sensor simulado en esta PC dieron la vuelta por Frankfurt y volvieron en **235 ms de media**, con una desviación de medio milisegundo sobre ocho mediciones. La arquitectura no cambió ni una línea respecto de la Actividad 1: solo cambió el valor de `MQTT_BROKER`. Eso es precisamente lo que hace útil al patrón publicador/suscriptor.
2. **La escalabilidad de MQTT es estructural, no una promesa.** `broker.hivemq.com` resuelve a seis direcciones, y en el experimento E cuatro conexiones simultáneas se repartieron sobre **tres nodos distintos del clúster** sin que ningún extremo se enterara ni se perdiera un mensaje: los tres suscriptores recibieron las cinco publicaciones. Un servidor WebSocket habría tenido que gestionar esas tres conexiones él mismo.
3. **WebSocket resuelve la interfaz, no la telemetría.** Una sola conexión sostenida 34 s reemplazó a las cuatro conexiones TCP completas que HTTP necesitó para cuatro lecturas, y la extensión `permessage-deflate` redujo cada lectura de 66 a 21 bytes. Es el transporte correcto para un panel en vivo, pero es punto a punto: no desacopla productores de consumidores.
4. **CoAP es el más eficiente y con diferencia.** Dos datagramas UDP y 255 bytes por lectura, frente a 14 tramas y 1.363 bytes de HTTP: **cinco veces menos tráfico y siete veces menos paquetes** para el mismo dato. Una petición `GET` cabe en 21 bytes. Para un dispositivo con batería, donde cada paquete transmitido es energía consumida, esa diferencia decide la autonomía.
5. **Salir a Internet cambia el análisis de las cuatro capas.** En capa 2 aparecen MACs reales y la del gateway en lugar de la del destino; en capa 3, IPs públicas, NAT y 9 saltos de router en lugar de `127.0.0.1`; en capa 4, un MSS de 1.460 en lugar de 65.495 y una latencia 10.000 veces mayor; en capa 7, un bróker que tarda segundos en contestar un CONNECT. Ninguna de estas cosas es visible en `localhost`, y una de ellas —el CONNACK tardío— provocó una pérdida de mensajes real que solo la captura de red permitió explicar.

6. **El payload sigue viajando en claro.** Todo lo anterior ocurrió sobre el puerto 1883, sin TLS: cualquiera en el camino lee `S-001` y sus temperaturas, y puede publicar en el mismo tópico. En un bróker público, además, el tópico es accesible para todo el mundo. La contramedida es la de la Actividad 1 —MQTT sobre TLS, puerto 8883, o `wss://` en el 8884 — más autenticación y control de acceso por tópico, que un bróker de pruebas gratuito no ofrece.